



Санкт-Петербургский государственный университет
Факультет прикладной математики – процессов управления

Параллельное программирование в Golang

Платонов Иван Николаевич, группа 24.M12-пу

Санкт-Петербург
2024

Введение: немного истории

- Праотцами идеи подхода Golang к организации конкурентности является CSP Хоара и guarded commands Дийкстры.
- Идея CSP (Communicating Sequential Processes) — одновременно выполняющиеся процессы не разделяют память, а взаимодействуют друг с другом посредством передачи сообщений через каналы.
- Языки, использующие похожую задумку:
 - ▶ Occam (May, 1983)
 - ▶ Erlang (Armstrong, 1986)
 - ▶ Newsqueak (Pike, 1988)
 - ▶ Concurrent ML (Reppy, 1993)
 - ▶ Alef (Winterbottom, 1995)
 - ▶ Limbo (Dorward, Pike, Winterbottom, 1996).

Горутина (Goroutine) — легковесный поток, который управляется планировщиком во время выполнения программы.

- Запускаются в общем адресном пространстве
- Имеют динамический стек (в отличие от потоков ОС)
- Размещаются на зеленых потоках

Горутины: примеры

```
func main() {  
    data := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}  
    parts := 4  
    part_length := len(data) / parts  
  
    for i := 0; i < len(data); i += part_length {  
        go func(l int, r int) {  
            for i := l; i < min(r, len(data)); i++ {  
                data[i]++  
            }  
        }(i, i+part_length)  
    }  
  
    fmt.Println("New array: ", data)  
}
```

Горутины: примеры

```
func main() {  
    data := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}  
    parts := 4  
    part_length := len(data) / parts  
  
    for i := 0; i < len(data); i += part_length {  
        go func(l int, r int) {  
            for i := l; i < min(r, len(data)); i++ {  
                data[i]++  
            }  
        }(i, i+part_length)  
    }  
  
    time.Sleep(1 * time.Second)  
    fmt.Println("New array: ", data)  
}
```

Подходит, если не волнует результат конкурентных операций

```
func main() {
    data := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}
    parts := 4
    part_length := len(data) / parts

    var wg sync.WaitGroup
    for i := 0; i < len(data); i += part_length {
        wg.Add(1)
        go func(l int, r int) {
            defer wg.Done()
            for i := l; i < min(r, len(data)); i++ {
                data[i]++
            }
        }(i, i+part_length)
    }

    wg.Wait()
    fmt.Println("New array: ", data)
}
```

Канал (Channel) — типизированный канал, который позволяет отправлять и читать из него данные.

- Буферизированный
 - ▶ Позволяет создать очередь сообщений
 - ▶ Не гарантирует синхронность
- Небуферизированный
 - ▶ Обеспечивает синхронность операций
 - ▶ Получатель и отправитель блокируются, пока не примут/не отдадут значение

Небуферизированный канал: пример

```
ch := make(chan int)

total := 0

var wg sync.WaitGroup
for i := 0; i < len(data); i += part_length {
    wg.Add(1)
    go func(l int, r int) {
        defer wg.Done()
        partial_sum := 0
        for i := l; i < min(r, len(data)); i++ {
            partial_sum += data[i]
        }
        ch <- partial_sum
    }(i, i+part_length)
}
go func() {
    for ps := range ch {
        total += ps
    }
}()

wg.Wait()
close(ch)
```

Небуферизированный канал: пример (исправленная версия)

```
ch := make(chan int)

total := 0

var wg sync.WaitGroup
for i := 0; i < len(data); i += part_length {
    wg.Add(1)
    go func(l int, r int) {
        defer wg.Done()
        partial_sum := 0
        for i := l; i < min(r, len(data)); i++ {
            partial_sum += data[i]
        }
        ch <- partial_sum
    }(i, i+part_length)
}

var wg_total sync.WaitGroup
wg_total.Add(1)
go func() {
    defer wg_total.Done()
    for ps := range ch {
        total += ps
    }
}()

wg.Wait()
close(ch)
wg_total.Wait()
```

Буферизированный канал: пример

```
ch := make(chan int, parts)

total := 0

var wg sync.WaitGroup
for i := 0; i < len(data); i += part_length {
    wg.Add(1)
    go func(l int, r int) {
        defer wg.Done()
        partial_sum := 0
        for i := l; i < min(r, len(data)); i++ {
            partial_sum += data[i]
        }
        ch <- partial_sum
    }(i, i+part_length)
}

wg.Wait()
close(ch)

for ps := range ch {
    total += ps
}
```

Небуферизированный канал: пример

```
func countSum(data []int, int l, int r, wg *sync.WaitGroup) <- chan int {
    ch := make(chan int)
    wg.Add(1)
    go func() {
        defer wg.Done()
        partial_sum := 0
        for i := l; i < min(r, len(data)); i++ {
            partial_sum += data[i]
        }
        ch <- partial_sum
    }()
    return ch
}

sumChan := make(chan int)
total := 0

var wg sync.WaitGroup
for i := 0; i < len(data); i += part_length {
    sumChan <- <-countSum(data, i, i+part_length, wg)
}

go func() {
    defer wg_total.Done()
    for ps := range sumChan {
        total += ps
    }
}()
}
```

Select + каналы: примеры

```
c := make(chan string)

for {
    select {
        case s := <-c:
            fmt.Println(s)
        case <-time.After(1 * time.Second):
            fmt.Println("You're too slow")
            return
    }
}
```

```
c := make(chan string)
timeout := time.After(5 * time.Second)

for {
    select {
        case s := <-c:
            fmt.Println(s)
        case <-timeout:
            fmt.Println("Time is over")
            return
    }
}
```

Select + каналы: примеры

```
c := make(chan int)
quit := make(chan bool)
for i := 0; i < 10; i++ {
    c <- i
}
quit <- true

for {
    select {
    case s := <-c:
        fmt.Println(s)
    case <-quit:
        fmt.Println("Job is done")
        return
    }
}
```

Классический подход к конкурентности используя memory access synchronization реализован в библиотеке sync.

- Mutex
- WaitGroup
- Once
- Pool
- Cond
- ...

Библиотека sync: Mutex

```
var count int
var lock sync.Mutex
increment := func() {
    lock.Lock()
    defer lock.Unlock()
    count++
}
decrement := func() {
    lock.Lock()
    defer lock.Unlock()
    count--
}
var arithmetic sync.WaitGroup
for i := 0; i <= 5; i++ { // Increment
    arithmetic.Add(1)
    go func() {
        defer arithmetic.Done()
        increment()
    }()
}
for i := 0; i <= 5; i++ { // Decrement
    arithmetic.Add(1)
    go func() {
        defer arithmetic.Done()
        decrement()
    }()
}
arithmetic.Wait()
```

Библиотека sync: Once

```
var count int
increment := func() {
    count++
}

var once sync.Once

var increments sync.WaitGroup
increments.Add(100)
for i := 0; i < 100; i++ {
    go func() {
        defer increments.Done()
        once.Do(increment)
    }()
}

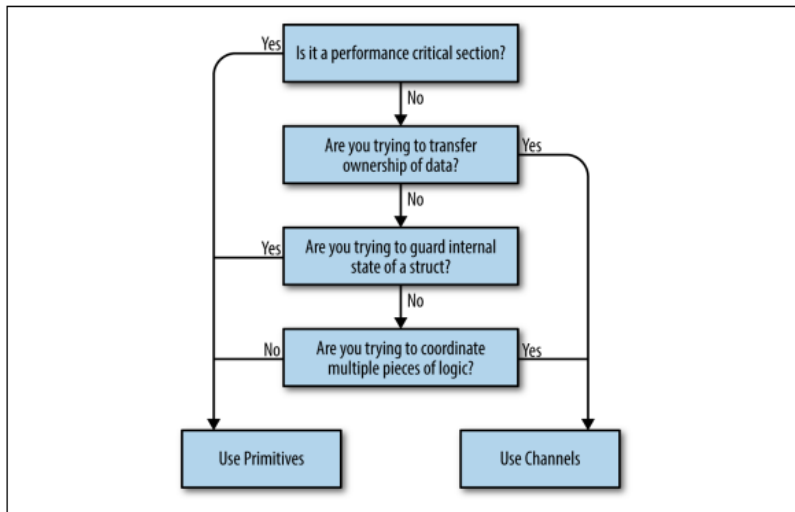
increments.Wait()
fmt.Printf("Count is %d\n", count)
```

```
var count int
increment := func() { count++ }
decrement := func() { count-- }

var once sync.Once
once.Do(increment)
once.Do(decrement)

fmt.Printf("Count: %d\n", count)
```

Выбор подхода к организации конкурентности



Выбор подхода к организации конкурентности

“ Regarding mutexes, the sync package implements them, but we hope Go programming style will encourage people to try higher-level techniques. In particular, consider structuring your program so that only one goroutine at a time is ever responsible for a particular piece of data. Do not communicate by sharing memory. Instead, share memory by communicating. ”

- Golang предлагает удобный подход к организации конкурентности, основанный на CSP
- Выполнение функций происходит в горутинах, которые размещаются на зеленых потоках
- Несмотря на все преимущества, в некоторых ситуациях не обойтись без классических примитив